BASE DE DATOS

Mishutec (Tienda de Componentes de Cómputo)

Presentación de la empresa

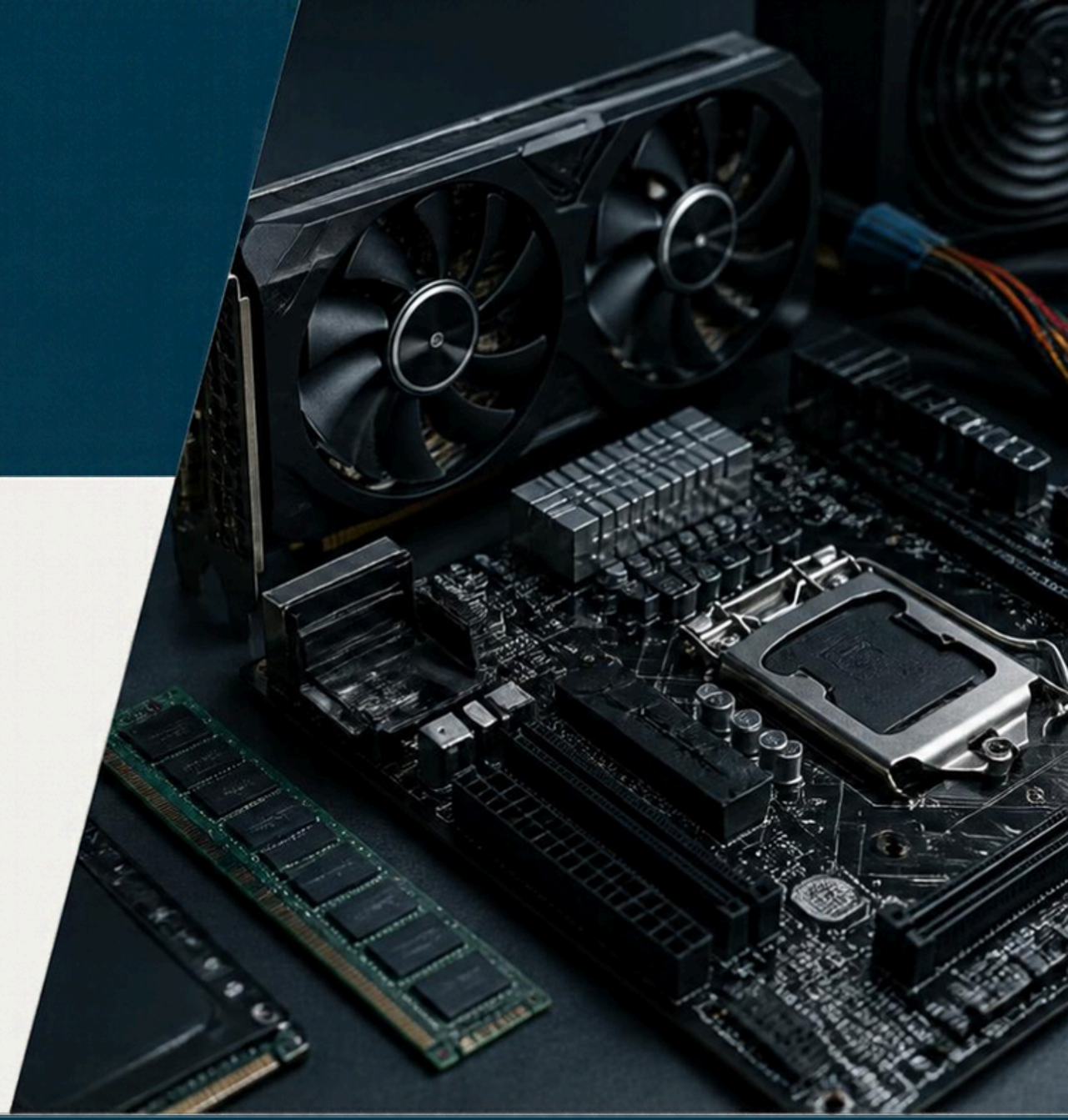
Mishutec (Tienda de Componentes de Cómputo)

Mishutec es una empresa especializada en la comercialización de componentes de cómputo.

Ofrecemos productos de calidad para ensamblar, actualizar y optimizar equipos.

Nuestro compromiso es brindar soluciones confiables con atención personalizada.

Trabajamos para satisfacer las necesidades de estudiantes, profesionales y empresas del sector.



Presentación de la empresa

Descripción de la actividad económica

Comercialización, distribución y venta al por menor y mayor de hardware, repuestos, piezas y accesorios tecnológicos para computadoras de escritorio y equipos, incluyendo procesadores, memorias RAM, unidades de estado sólido, tarjetas gráficas, fuentes de poder y gabinetes.

Presentación de la empresa

Realidad problemática

Control manual o desarticulado de inventarios, ventas, garantías y devoluciones

Falta de información en tiempo real y procesos dispersos sin integración

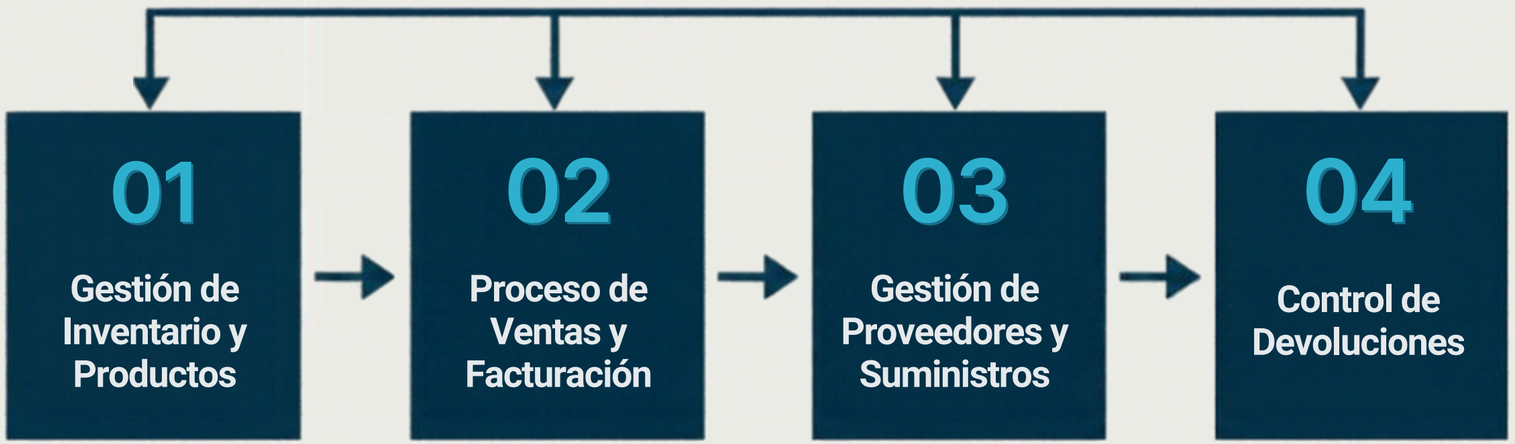
Errores de stock y desabastecimientos

Pérdidas por productos defectuosos

Retrasos en la atención al cliente

Identificación de los procesos de la empresa

Procesos que abarca la Base de Datos



Gestión de Inventario y Productos

Control de entradas, salidas y movimientos de componentes de PC.

Proceso de Ventas y Facturación

Emisión de comprobantes (boletas, facturas) y registro de múltiples métodos de pago.

Gestión de Proveedores y Suministros

Registro de compras y abastecimiento de mercadería.

Control de Devoluciones

Gestión de garantías por fallas de fábrica con motivos documentados.

Identificación de la Propuesta

Objetivos del proyecto

OBJETIVO GENERAL

Desarrollar un sistema para mejorar la gestión de productos y ventas mediante una base de datos.

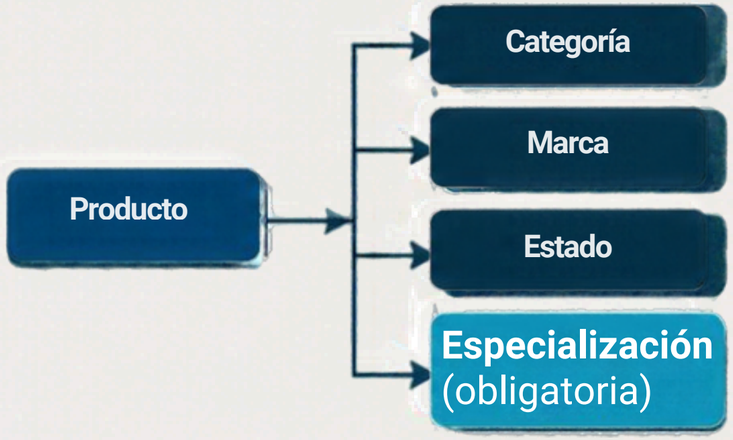
OBJETIVOS ESPECÍFICOS

- Registrar y administrar productos
- Registrar y almacenar ventas
- Consultar ventas de forma eficiente

Diseño de la Propuesta

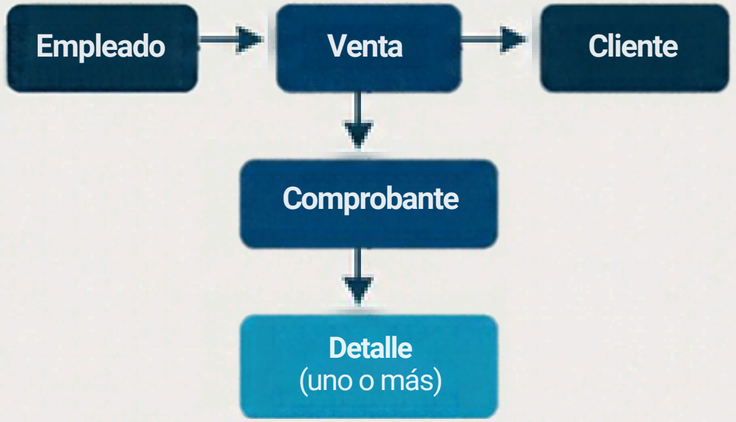
Reglas del negocio para la Base de Datos

1 Gestión de productos



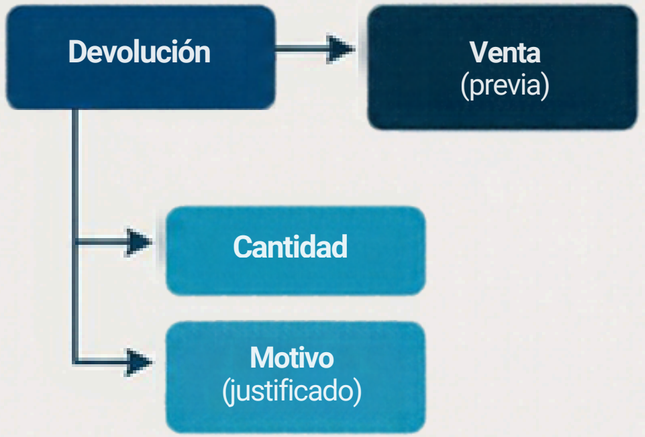
Cada producto debe registrarse obligatoriamente con una categoría específica: **CPU, RAM, SSD, tarjeta gráfica, fuente o gabinete.**
El stock de un producto no puede ser 0 o

2 Registro de ventas



Cada venta es realizada por un empleado a un cliente, genera un **comprobante** y contiene uno o más detalles.
La **cantidad** de cada detalle no puede ser 0 o menor.

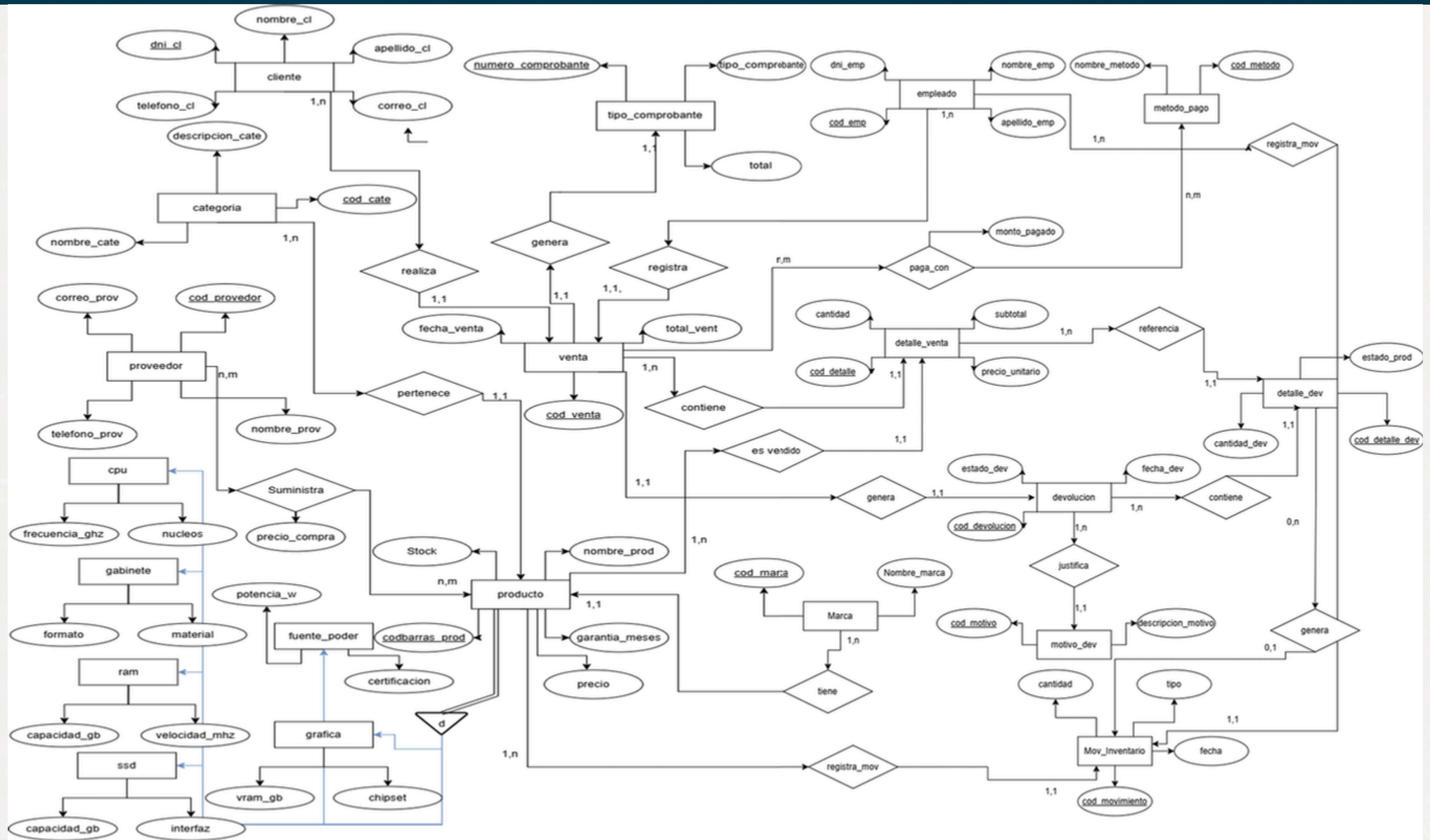
3 Gestión de devoluciones



Toda devolución está vinculada a una venta previa e incluye la **cantidad** devuelta y un **motivo** justificado.

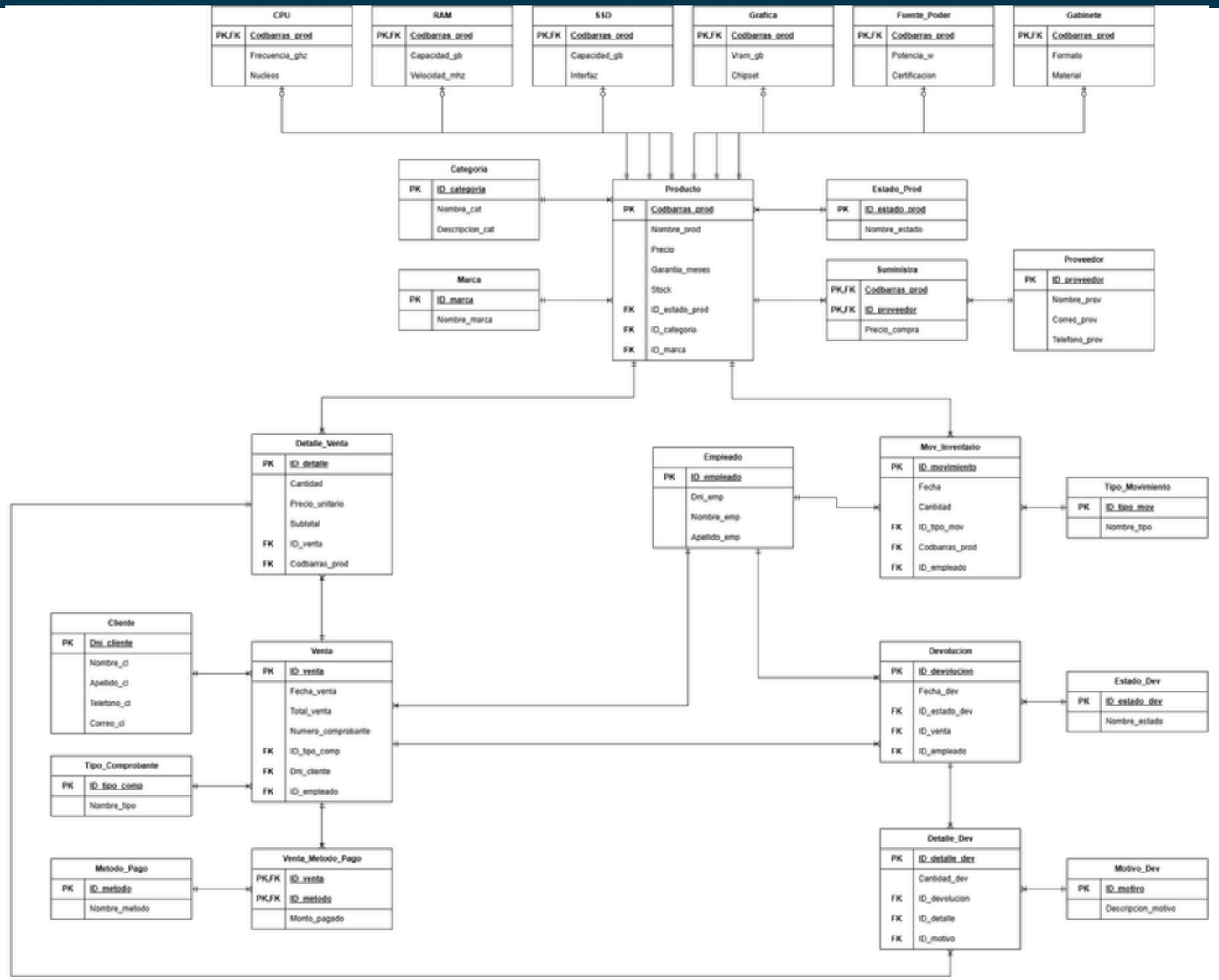
Diseño Conceptual

Diagrama Entidad Relación



Diseño Lógico

Diagrama Relacional (con 1FN, 2FN y FN)



Diseño fisico

Sentencias ddl

Create

Mostrar la creacion de las tablas y la relacion entre tablas a traves de las PK y FK

```
Create database MishutecT2
GO

USE MishutecT2;
GO

-- Tablas catálogo
CREATE TABLE dbo.Marca (
    ID_marca INT IDENTITY(1,1) PRIMARY KEY,
    Nombre_marca VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE dbo.Categoria (
    ID_categoria INT IDENTITY(1,1) PRIMARY KEY,
    Nombre_cat VARCHAR(50) NOT NULL UNIQUE,
    Descripcion_cat VARCHAR(200)
);

CREATE TABLE dbo.Estado_Prod (
    ID_estado_prod INT IDENTITY(1,1) PRIMARY KEY,
    Estado VARCHAR(30) NOT NULL UNIQUE
);
```

```
CREATE TABLE dbo.Producto (
    Codbarras_prod VARCHAR(13) PRIMARY KEY,
    Nombre_prod VARCHAR(120) NOT NULL,
    Precio DECIMAL(10,2) NOT NULL,
    Garantia_meses INT NOT NULL,
    Stock INT NOT NULL,
    ID_estado_prod INT NOT NULL,
    ID_categoria INT NOT NULL,
    ID_marca INT NOT NULL,
    CONSTRAINT CK_Producto_Stock CHECK (Stock >= 0),
    CONSTRAINT FK_Producto_Estado FOREIGN KEY (ID_estado_prod) REFERENCES dbo.Estado_Prod(ID_estado_prod),
    CONSTRAINT FK_Producto_Categoria FOREIGN KEY (ID_categoria) REFERENCES dbo.Categoria(ID_categoria),
    CONSTRAINT FK_Producto_Marca FOREIGN KEY (ID_marca) REFERENCES dbo.Marca(ID_marca)
);
```

Diseño fisico

Sentencias ddl

Truncate

Elimina todos los registros de una tabla por completo, dejando la estructura de la tabla

```
INSERT INTO dbo.Mov_Inventario (Fecha, Cantidad, ID_tipo_mov, Codbarras_prod, ID_empleado)
VALUES ('2026-08-25', 700, 1, '7750000000001', 1),    -- cantidad mal digitada
       ('2026-08-25', 900, 1, '7750000000002', 1),    -- cantidad mal digitada
       ('2026-08-25',  50, 2, '7750000000099', 1);    -- código de barras inexistente
```

```
SELECT * FROM dbo.Mov_Inventario;
```

Resultados | Mensajes

	ID_movimiento	Fecha	Cantidad	ID_tipo_mov	Codbarras_prod	ID_empleado
1	1	2026-08-25	700	1	7750000000001	1
2	2	2026-08-25	900	1	7750000000002	1
3	3	2026-08-25	50	2	7750000000099	1

```
276
277   TRUNCATE TABLE dbo.Mov_Inventario;
278
279   SELECT * FROM dbo.Mov_Inventario;      -- Después del TRUNCATE: tabla vacía
280   GO
281
```

Resultados | Mensajes

ID_movimiento	Fecha	Cantidad	ID_tipo_mov	Codbarras_prod	ID_empleado

Diseño fisico

Sentencias ddl

Alter

Modificar la estructura de un objeto existente (como una tabla, vista o procedimiento) sin necesidad de borrarlo y crearlo de nuevo

```
CREATE TABLE dbo.Cliente (
    Dni_cliente VARCHAR(8) PRIMARY KEY,
    Nombre_cl VARCHAR(60) NOT NULL,
    Apellido_cl VARCHAR(60) NOT NULL,
    Telefono_cl VARCHAR(15)
);
```

```
-- ADD (columna): agregar la columna Correo_cl que faltó en Cliente
ALTER TABLE dbo.Cliente ADD Correo_cl VARCHAR(100) not NULL;
GO
```

```
CREATE TABLE dbo.Mov_Inventario (
    ID_movimiento INT IDENTITY(1,1) NOT NULL,
    Fecha DATE NOT NULL,
    Cantidad INT NOT NULL,
    ID_tipo_mov INT NOT NULL,
    Codbarras_prod VARCHAR(13) NOT NULL,
    ID_empleado INT NOT NULL,
    CONSTRAINT CK_Mov_Cantidad CHECK (Cantidad > 0)
);
```

```
-- ADD CONSTRAINT PRIMARY KEY: Mov_Inventario no tenía clave primaria
ALTER TABLE dbo.Mov_Inventario
ADD CONSTRAINT PK_Mov_Inventario PRIMARY KEY (ID_movimiento);
GO
```

```
CREATE TABLE dbo.Proveedor (
    ID_proveedor INT IDENTITY(1,1) PRIMARY KEY,
    Nombre_prov VARCHAR(100) NULL,
    Corero_prov VARCHAR(100),
    Telefono_prov VARCHAR(15)
);
```

```
ALTER TABLE dbo.Proveedor ALTER COLUMN Nombre_prov VARCHAR(100) NOT NULL;
GO
ALTER TABLE dbo.Proveedor ADD Correo_prov VARCHAR(100) Not NULL;
GO
```

```
ALTER TABLE dbo.Proveedor DROP COLUMN Corero_prov;
GO
```

Diseño fisico

Sentencias DDL

Drop

Sirve para eliminar por completo un objeto de la base de datos (como una tabla, una base de datos, un índice o una vista)

```
388  SELECT TABLE_NAME AS Tabla_Antes_del_DROP
389  FROM INFORMATION_SCHEMA.TABLES
390  WHERE TABLE_NAME LIKE 'Venta_Met%';
391  GO
392
393  DROP TABLE dbo.Venta_Metdo_Pago;
```

132 % ⊗ 25 ⚠ 0

Resultados | Mensajes

	Tabla_Antes_del_DROP
1	Venta_Metdo_Pago

```
397  CREATE TABLE dbo.Venta_Metodo_Pago (
398      ID_venta INT NOT NULL,
399      ID_metodo INT NOT NULL,
400      Monto_pagado DECIMAL(10,2) NOT NULL,
401      CONSTRAINT PK_Venta_Metodo_Pago PRIMARY KEY (ID_venta, ID_metodo),
402      CONSTRAINT CK_VMP_Monto CHECK (Monto_pagado > 0),
403      CONSTRAINT FK_VMP_Venta FOREIGN KEY (ID_venta) REFERENCES dbo.Venta(ID_venta),
404      CONSTRAINT FK_VMP_Metodo FOREIGN KEY (ID_metodo) REFERENCES dbo.Metodo_Pago(ID_metodo)
405  );
406  GO
407
408  SELECT TABLE_NAME AS Tabla_Despues_del_DROP
409  FROM INFORMATION_SCHEMA.TABLES
410  WHERE TABLE_NAME LIKE 'Venta_Met%';
411  GO
```

Resultados | Mensajes

	Tabla_Despues_del_DROP
1	Venta_Metodo_Pago

Diseño fisico

Sentencias DML

Insert

Se utiliza para agregar nuevos registros o filas a una tabla en una base de datos

ID_marca	Nombre_marca

ID_categoria	Nombre_cat	Descripcion_cat

ID_estado_prod	Estado

ID_proveedor	Nombre_prov	Telefono_prov	Correo_prov

```
INSERT INTO dbo.Marca (Nombre_marca)
VALUES ('Kingston'), ('Corsair'), ('Samsung'), ('AMD'), ('Intel'), ('ASUS'), ('Cooler Master');

INSERT INTO dbo.Categoria (Nombre_cat, Descripcion_cat)
VALUES ('RAM', 'Memorias RAM'),
       ('SSD', 'Unidades de estado sólido'),
       ('CPU', 'Procesadores'),
       ('Grafica', 'Tarjetas gráficas'),
       ('Fuente de poder', 'Fuentes de alimentación para PC'),
       ('Gabinete', 'Cases y gabinetes para PC');

INSERT INTO dbo.Estado_Prod (Estado)
VALUES ('Disponible'), ('Agotado'), ('Descontinuado');

INSERT INTO dbo.Proveedor (Nombre_prov, Correo_prov, Telefono_prov)
VALUES ('Tecno Import Perú SAC', 'ventas@tecnoimport.pe', '014567890'),
       ('Distribuidora Andina de Cómputo', 'contacto@andinacomputo.pe', '014123456'),
       ('Global Hardware SAC', 'pedidos@globalhardware.pe', '017894561');
```

	ID_marca	Nombre_marca
1	4	AMD
2	6	ASUS
3	7	Cooler Master
4	2	Corsair
5	5	Intel
6	1	Kingston
7	3	Samsung

	ID_categoria	Nombre_cat	Descripcion_cat
1	1	RAM	Memorias RAM
2	2	SSD	Unidades de estado sólido
3	3	CPU	Procesadores
4	4	Grafica	Tarjetas gráficas
5	5	Fuente de poder	Fuentes de alimentación para PC
6	6	Gabinete	Cases y gabinetes para PC

	ID_estado_prod	Estado
1	2	Agotado
2	3	Descontinuado
3	1	Disponible

	ID_proveedor	Nombre_prov	Telefono_prov	Correo_prov
1	1	Tecno Import Perú SAC	014567890	ventas@tecnoimport.pe
2	2	Distribuidora Andina de Cómputo	014123456	contacto@andinacomputo.pe
3	3	Global Hardware SAC	017894561	pedidos@globalhardware.pe

Diseño fisico

Sentencias DML

Delete

Se utiliza para eliminar filas existentes de una tabla, conservando intactas su estructura, columnas e índices

	Dni_cliente	Nombre_cl	Apellido_cl	Telefono_cl	Correo_cl
1	70231456	Luis	Rojas Medina	986112233	luis.rojas@outlook.com
2	71458963	Pedro	Huamán Soto	965874123	pedro.huaman@gmail.com
3	72145809	Ana	Torres Quispe	987654321	ana.torres@gmail.com
4	72145890	Ana	Torres Quispe	987654321	ana.torres@gmail.com
5	73654128	Lucía	Flores Rivas	917233318	lucia.flores@gmail.com
6	75896412	Mia	Salas Campos	98456123	mia.salas@gmail.com

```
DELETE FROM dbo.Cliente
WHERE Dni_cliente = '72145809';
```

	Dni_cliente	Nombre_cl	Apellido_cl	Telefono_cl	Correo_cl
1	70231456	Luis	Rojas Medina	986112233	luis.rojas@outlook.com
2	71458963	Pedro	Huamán Soto	965874123	pedro.huaman@gmail.com
3	72145890	Ana	Torres Quispe	987654321	ana.torres@gmail.com
4	73654128	Lucía	Flores Rivas	917233318	lucia.flores@gmail.com
5	75896412	Mia	Salas Campos	98456123	mia.salas@gmail.com

Diseño fisico

Sentencias DML

Update

Se utiliza para modificar o actualizar los datos que ya existen en una tabla

```
INSERT INTO dbo.Producto
(Codbarras_prod, Nombre_prod, Precio, Garantia_meses, Stock, ID_estado_prod, ID_categoria, ID_marca)
SELECT v.cod, v.nombre, v.precio, v.garantia, v.stock, e.ID_estado_prod, c.ID_categoria, m.ID_marca
FROM (VALUES
    ('7750000000001', 'Kingston Fury Beast 16GB DDR4 3200', 180.00, 24, 15, 'Disponible', 'RAM',            'Kingston'),
    ('7750000000002', 'Corsair Vengeance LPX 32GB DDR4 3600', 320.00, 36,  8, 'Disponible', 'RAM',            'Corsair'),
    ('7750000000003', 'Kingston NV2 1TB NVMe',                260.00, 24, 10, 'Disponible', 'SSD',            'Kingston'),
    ('7750000000004', 'Samsung 980 500GB NVMe',                19.00, 36, 12, 'Disponible', 'SSD',            'Samsung'),
    ('7750000000005', 'AMD Ryzen 7 5700X',                    950.00, 36,  5, 'Disponible', 'CPU',            'AMD'),
    ('7750000000006', 'Intel Core i5-12400F',                 620.00, 36,  6, 'Agotado',    'CPU',            'Intel'),
    ('7750000000007', 'ASUS Dual GeForce RTX 4060 8GB',      1450.00, 36, 40, 'Disponible', 'Grafica',        'ASUS'),
    ('7750000000008', 'Corsair CV650 650W 80 Plus Bronze',    280.00, 36,  7, 'Disponible', 'Fuente de poder', 'Corsair'),
    ('7750000000009', 'Cooler Master MasterBox Q300L',        240.00, 12,  6, 'Disponible', 'Gabinete',        'Cooler Master')
) AS v(cod, nombre, precio, garantia, stock, estado, cat, marca)
JOIN dbo.Estado_Prod e ON e.Estado = v.estado
JOIN dbo.Categoria  c ON c.Nombre_cat = v.cat
JOIN dbo.Marca      m ON m.Nombre_marca = v.marca;
```

	Codbarras_prod	Nombre_prod	Precio	Garantia_meses	Stock	ID_estado_prod	ID_categoria	ID_marca
1	7750000000001	Kingston Fury Beast 16GB DDR4 3200	180.00	24	15	1	1	1
2	7750000000002	Corsair Vengeance LPX 32GB DDR4 3600	320.00	36	8	1	1	2
3	7750000000003	Kingston NV2 1TB NVMe	260.00	24	10	1	2	1
4	7750000000004	Samsung 980 500GB NVMe	19.00	36	12	1	2	3
5	7750000000005	AMD Ryzen 7 5700X	950.00	36	5	1	3	4
6	7750000000006	Intel Core i5-12400F	620.00	36	6	2	3	5
7	7750000000007	ASUS Dual GeForce RTX 4060 8GB	1450.00	36	40	1	4	6
8	7750000000008	Corsair CV650 650W 80 Plus Bronze	280.00	36	7	1	5	2
9	7750000000009	Cooler Master MasterBox Q300L	240.00	12	6	1	6	7
10	7750000000010	Samsung 870 EVO 1TB SATA	330.00	36	0	2	2	3

```
-- Corregir el precio del Samsung 980 500GB (19.00 -> 190.00)
UPDATE dbo.Producto
SET Precio = 190.00
WHERE Codbarras_prod = '7750000000004';

-- Corregir el stock de la ASUS RTX 4060 (40 -> 4)
UPDATE dbo.Producto
SET Stock = 4
WHERE Codbarras_prod = '7750000000007';

-- Corregir el estado del Intel i5 (tiene stock, por lo tanto está Disponible)
UPDATE dbo.Producto
SET ID_estado_prod = (SELECT ID_estado_prod FROM dbo.Estado_Prod WHERE Estado = 'Disponible')
WHERE Codbarras_prod = '7750000000006'
  AND Stock > 0;
```

	Codbarras_prod	Nombre_prod	Precio	Garantia_meses	Stock	Estado_Producto	Categoria	Marca
1	7750000000001	Kingston Fury Beast 16GB DDR4 3200	180.00	24	15	Disponible	RAM	Kingston
2	7750000000002	Corsair Vengeance LPX 32GB DDR4 3600	320.00	36	8	Disponible	RAM	Corsair
3	7750000000003	Kingston NV2 1TB NVMe	260.00	24	10	Disponible	SSD	Kingston
4	7750000000004	Samsung 980 500GB NVMe	190.00	36	12	Disponible	SSD	Samsung
5	7750000000005	AMD Ryzen 7 5700X	950.00	36	5	Disponible	CPU	AMD
6	7750000000006	Intel Core i5-12400F	620.00	36	6	Disponible	CPU	Intel
7	7750000000007	ASUS Dual GeForce RTX 4060 8GB	1450.00	36	4	Disponible	Grafica	ASUS
8	7750000000008	Corsair CV650 650W 80 Plus Bronze	280.00	36	7	Disponible	Fuente de poder	Corsair
9	7750000000009	Cooler Master MasterBox Q300L	240.00	12	6	Disponible	Gabinete	Cooler Master
10	7750000000010	Samsung 870 EVO 1TB SATA	330.00	36	0	Agotado	SSD	Samsung

Diseño fisico

Sentencias DML

Consultas

Se utiliza para recuperar y visualizar datos de una o varias tablas en una base de datos

```sql
-- Consulta 1: operadores relacionales, lógicos, WHERE y ORDER BY.
SELECT Nombre_prod, Precio, Stock
FROM dbo.Producto
WHERE Precio >= 250 AND Stock > 0
ORDER BY Precio DESC;
```

	Nombre_prod	Precio	Stock
1	ASUS Dual GeForce RTX 4060 8GB	1450.00	4
2	AMD Ryzen 7 5700X	950.00	5
3	Intel Core i5-12400F	620.00	6
4	Corsair Vengeance LPX 32GB DDR4 3600	320.00	8
5	Corsair CV650 650W 80 Plus Bronze	280.00	7
6	Kingston NV2 1TB NVMe	260.00	10

```sql
-- Consulta 2: operadores aritméticos, alias y filtro OR.
SELECT Nombre_prod, Precio,
       Precio * 1.18 AS Precio_con_IGV,
       Stock,
       Precio * Stock AS Valor_stock
FROM dbo.Producto
WHERE ID_categoria = (SELECT ID_categoria FROM dbo.Categoria WHERE Nombre_cat = 'RAM')
   OR Precio > 900
ORDER BY Nombre_prod ASC;
```

	Nombre_prod	Precio	Stock	Valor_stock	Precio_con_IGV
1	AMD Ryzen 7 5700X	950.00	5	4750.00	1121.0000
2	ASUS Dual GeForce RTX 4060 8GB	1450.00	4	5800.00	1711.0000
3	Corsair Vengeance LPX 32GB DDR4 3600	320.00	8	2560.00	377.6000
4	Kingston Fury Beast 16GB DDR4 3200	180.00	15	2700.00	212.4000

Diseño fisico

Sentencias DML

Mono Registro

Consulta anidada que devuelve un único valor

```
SELECT Nombre_prod, Precio
FROM dbo.Producto
WHERE Precio > (SELECT AVG(Precio) FROM dbo.Producto);
```

	Nombre_prod	Precio
1	AMD Ryzen 7 5700X	950.00
2	Intel Core i5-12400F	620.00
3	ASUS Dual GeForce RTX 4060 8GB	1450.00

```
SELECT Nombre_prod, Precio
FROM dbo.Producto
WHERE Precio = (SELECT MAX(Precio) FROM dbo.Producto);
```

	Nombre_prod	Precio
1	ASUS Dual GeForce RTX 4060 8GB	1450.00

```
SELECT Nombre_prod, Precio
FROM dbo.Producto
WHERE ID_marca = (SELECT ID_marca FROM dbo.Marca WHERE Nombre_marca = 'Kingston');
```

	Nombre_prod	Precio
1	Kingston Fury Beast 16GB DDR4 3200	180.00
2	Kingston NV2 1TB NVMe	260.00

Diseño fisico

Sentencias DML

Multi Registro

Capacidad de procesar varias filas de datos a la vez en una sola instrucción o sentencia, en lugar de hacerlo una por una

```sql
SELECT Nombre_prod, Precio
FROM dbo.Producto
WHERE ID_categoria IN (
    SELECT ID_categoria
    FROM dbo.Categoria
    WHERE Nombre_cat IN ('RAM', 'SSD')
);
```

	Nombre_prod	Precio
1	Kingston Fury Beast 16GB DDR4 3200	180.00
2	Corsair Vengeance LPX 32GB DDR4 3600	320.00
3	Kingston NV2 1TB NVMe	260.00
4	Samsung 980 500GB NVMe	190.00
5	Samsung 870 EVO 1TB SATA	330.00

```sql
SELECT Nombre_prod, Precio
FROM dbo.Producto
WHERE Precio > ANY (
    SELECT p2.Precio
    FROM dbo.Producto p2
    INNER JOIN dbo.Categoria c2 ON c2.ID_categoria = p2.ID_categoria
    WHERE c2.Nombre_cat = 'RAM'
);
```

	Nombre_prod	Precio
1	Corsair Vengeance LPX 32GB DDR4 3600	320.00
2	Kingston NV2 1TB NVMe	260.00
3	Samsung 980 500GB NVMe	190.00
4	AMD Ryzen 7 5700X	950.00
5	Intel Core i5-12400F	620.00
6	ASUS Dual GeForce RTX 4060 8GB	1450.00
7	Corsair CV650 650W 80 Plus Bronze	280.00
8	Cooler Master MasterBox Q300L	240.00
9	Samsung 870 EVO 1TB SATA	330.00

```sql
SELECT Nombre_prod, Precio
FROM dbo.Producto
WHERE Precio > ALL (
    SELECT p2.Precio
    FROM dbo.Producto p2
    INNER JOIN dbo.Categoria c2 ON c2.ID_categoria = p2.ID_categoria
    WHERE c2.Nombre_cat = 'SSD'
);
```

	Nombre_prod	Precio
1	AMD Ryzen 7 5700X	950.00
2	Intel Core i5-12400F	620.00
3	ASUS Dual GeForc...	1450...

Diseño fisico

Sentencias DML

Multi Columna

Realizar operaciones, consultas, restricciones o agrupamientos que involucran a dos o más columnas al mismo tiempo

```sql
SELECT p.Nombre_prod, p.ID_marca, p.ID_categoria, p.Precio
FROM dbo.Producto p
WHERE EXISTS (
    SELECT 1
    FROM dbo.Producto x
    WHERE x.ID_marca = p.ID_marca
      AND x.ID_categoria = p.ID_categoria
      AND x.Precio > 500
);
```

	Nombre_prod	ID_marca	ID_categoria	Precio
1	Kingston Fury Beast 16GB DDR4 3200	1	1	180.00
2	Kingston Fury Renegade 32GB	1	1	550.00
3	AMD Ryzen 7 5700X	4	3	950.00
4	Intel Core i5-12400F	5	3	620.00
5	ASUS Dual GeForce RTX 4060 8GB	6	4	1450.00
6	Intel Core i3-12100F	5	3	240.00

```sql
SELECT p.Nombre_prod, p.ID_categoria, p.Precio
FROM dbo.Producto p
INNER JOIN (
    SELECT ID_categoria, MAX(Precio) AS Precio_max
    FROM dbo.Producto
    GROUP BY ID_categoria
) mx ON mx.ID_categoria = p.ID_categoria AND mx.Precio_max = p.Precio;
```

	Nombre_prod	ID_categoria	Precio
1	Cooler Master MasterBox Q300L	6	240.00
2	Corsair CV650 650W 80 Plus Bronze	5	280.00
3	ASUS Dual GeForce RTX 4060 8GB	4	1450.00
4	AMD Ryzen 7 5700X	3	950.00
5	Samsung 870 EVO 1TB SATA	2	330.00
6	Corsair Vengeance LPX 32GB DD...	1	320.00

```sql
SELECT
    d.ID_venta,
    d.Codbarras_prod,
    p.Nombre_prod,
    d.Cantidad
FROM dbo.Detalle_Venta d
INNER JOIN (
    SELECT ID_venta, MAX(Cantidad) AS Cant_max
    FROM dbo.Detalle_Venta
    GROUP BY ID_venta
) mx ON mx.ID_venta = d.ID_venta AND mx.Cant_max = d.Cantidad
INNER JOIN dbo.Producto p
    ON p.Codbarras_prod = d.Codbarras_prod;
```

	ID_venta	Codbarras_prod	Nombre_prod	Cantidad
1	5	7750000000004	Samsung 980 500GB NVMe	4
2	4	7750000000007	ASUS Dual GeForce RTX 4060 8GB	3
3	4	7750000000008	Corsair CV650 650W 80 Plus Bronze	3
4	3	7750000000003	Kingston NV2 1TB NVMe	1
5	2	7750000000005	AMD Ryzen 7 5700X	5
6	1	7750000000001	Kingston Fury Beast 16GB DDR4 3200	2

Diseño fisico

Sentencias DML

INNER JOIN

El INNER JOIN permite unir información de dos o más tablas relacionadas. Por ejemplo, podemos consultar qué cliente realizó una venta o qué productos se vendieron.

```sql
-- INNER JOIN 1: productos con marca y categoria.
SELECT p.Codbarras_prod, p.Nombre_prod, m.Nombre_marca, c.Nombre_cat, p.Pre
FROM dbo.Producto p
INNER JOIN dbo.Marca m ON p.ID_marca = m.ID_marca
INNER JOIN dbo.Categoria c ON p.ID_categoria = c.ID_categoria
ORDER BY m.Nombre_marca, p.Nombre_prod;

-- INNER JOIN 2: ventas con cliente y empleado.
SELECT v.Numero_comprobante, v.Fecha_venta, v.Total_venta,
       cl.Nombre_cl, cl.Apellido_cl, e.Nombre_emp, e.Apellido_emp
FROM dbo.Venta v
INNER JOIN dbo.Cliente cl ON v.Dni_cliente = cl.Dni_cliente
INNER JOIN dbo.Empleado e ON v.ID_empleado = e.ID_empleado
ORDER BY v.Fecha_venta DESC;

-- INNER JOIN 3: detalle de venta con nombre del producto.
SELECT v.Numero_comprobante, p.Nombre_prod, dv.Cantidad,
       dv.Precio_unitario, dv.Subtotal
FROM dbo.Detalle_Venta dv
INNER JOIN dbo.Venta v ON dv.ID_venta = v.ID_venta
INNER JOIN dbo.Producto p ON dv.Codbarras_prod = p.Codbarras_prod
ORDER BY v.Numero_comprobante;
```

	Codbarras_prod	Nombre_prod	Nombre_marca	Nombre_cat	Precio
1	7750000000005	AMD Ryzen 7 5700X	AMD	CPU	950.00
2	7750000000007	ASUS Dual GeForce RTX 4060 8GB	ASUS	Grafica	1450.00
3	7750000000008	Corsair CV650 650W 80 Plus Bronze	Corsair	Fuente de...	280.00
4	7750000000009	Intel Core i3-12100F	Intel	CPU	240.00
5	7750000000006	Intel Core i5-12400F	Intel	CPU	620.00
6	7750000000001	Kingston Fury Beast 16GB DDR4 3...	Kingston	RAM	180.00
7	7750000000002	Kingston Fury Renegade 32GB	Kingston	RAM	550.00
8	7750000000003	Samsung 870 EVO 1TB SATA	Samsung	SSD	260.00

	Numero_comprobante	Fecha_venta	Total_venta	Nombre_cl	Apellido_cl	Nombre_emp	Apellido_emp
1	B001-000004	2026-10-05	510.00	Pedro	Huamán Soto	Jorge	Mendoza Cruz
2	B001-000003	2026-10-01	1730.00	Ana	Torres Quispe	Rosa	Paredes Luna
3	B001-000002	2026-09-22	260.00	Mia	Salas Campos	Carlos	Vega Ramos
4	F001-000001	2026-09-10	1190.00	Luis	Rojas Medina	Rosa	Paredes Luna
5	B001-000001	2026-09-01	360.00	Ana	Torres Quispe	Carlos	Vega Ramos

	Numero_comprobante	Nombre_prod	Cantidad	Precio_unitario	Subtotal
1	B001-000001	Kingston Fury Beast 16GB DDR4 3200	2	180.00	360.00
2	B001-000002	Samsung 870 EVO 1TB SATA	1	260.00	260.00
3	B001-000003	ASUS Dual GeForce RTX 4060 8GB	3	1450.00	4350.00
4	B001-000003	Corsair CV650 650W 80 Plus Bronze	3	280.00	840.00
5	B001-000004	Kingston Fury Renegade 32GB	1	320.00	320.00
6	B001-000004	Samsung 980 500GB NVMe	4	190.00	760.00
7	F001-000001	AMD Ryzen 7 5700X	5	950.00	4750.00
8	F001-000001	Intel Core i3-12100F	2	240.00	480.00

Diseño fisico

Sentencias DML

INNER JOIN

El INNER JOIN permite unir información de dos o más tablas relacionadas. Por ejemplo, podemos consultar qué cliente realizó una venta o qué productos se vendieron.

```sql
-- INNER JOIN 4: proveedores que suministran productos.
SELECT pr.Nombre_prov, p.Nombre_prod, s.Precio_compra
FROM dbo.Suministra s
INNER JOIN dbo.Proveedor pr ON s.ID_proveedor = pr.ID_proveedor
INNER JOIN dbo.Producto p ON s.Codbarras_prod = p.Codbarras_prod
ORDER BY pr.Nombre_prov;
```

	Nombre_prov	Nombre_prod	Precio_compra
1	Distribuidora Andina de Cómputo	Kingston Fury Beast 16GB DDR4 3200	135.00
2	Distribuidora Andina de Cómputo	Kingston Fury Renegade 32GB	245.00
3	Distribuidora Andina de Cómputo	ASUS Dual GeForce RTX 4060 8GB	1200.00
4	Global Hardware SAC	Samsung 980 500GB NVMe	140.00
5	Global Hardware SAC	Intel Core i5-12400F	500.00
6	Global Hardware SAC	Intel Core i3-12100F	170.00
7	Tecno Import Perú SAC	Kingston Fury Beast 16GB DDR4 3200	130.00
8	Tecno Import Perú SAC	Samsung 870 EVO 1TB SATA	200.00
9	Tecno Import Perú SAC	AMD Ryzen 7 5700X	780.00
10	Tecno Import Perú SAC	Corsair CV650 650W 80 Plus Bronze	215.00

```sql
-- INNER JOIN 7: inventario con tipo de movimiento, producto y empleado.
SELECT mi.Fecha, tm.Nombre_tipo, p.Nombre_prod, mi.Cantidad,
       e.Nombre_emp, e.Apellido_emp
FROM dbo.Mov_Inventario mi
INNER JOIN dbo.Tipo_Movimiento tm ON mi.ID_tipo_mov = tm.ID_tipo_mov
INNER JOIN dbo.Producto p ON mi.Codbarras_prod = p.Codbarras_prod
INNER JOIN dbo.Empleado e ON mi.ID_empleado = e.ID_empleado
ORDER BY mi.Fecha DESC;
```

	Fecha	Nombre_tipo	Nombre_prod	Cantidad	Nombre_emp	Apellido_emp
1	2026-10-05	Salida	Kingston Fury Renegade 32GB	1	Jorge	Mendoza Cruz
2	2026-10-05	Salida	Samsung 980 500GB NVMe	1	Jorge	Mendoza Cruz
3	2026-10-01	Salida	ASUS Dual GeForce RTX 4060 8GB	1	Rosa	Paredes Luna
4	2026-10-01	Salida	Corsair CV650 650W 80 Plus Bronze	1	Rosa	Paredes Luna
5	2026-09-22	Salida	Samsung 870 EVO 1TB SATA	1	Carlos	Vega Ramos
6	2026-09-10	Salida	AMD Ryzen 7 5700X	1	Rosa	Paredes Luna
7	2026-09-10	Salida	Intel Core i3-12100F	1	Rosa	Paredes Luna
8	2026-09-01	Salida	Kingston Fury Beast 16GB DDR4 3200	2	Carlos	Vega Ramos
9	2026-08-25	Entrada	Kingston Fury Beast 16GB DDR4 3200	7	Carlos	Vega Ramos
10	2026-08-25	Entrada	Kingston Fury Renegade 32GB	9	Carlos	Vega Ramos
11	2026-08-25	Entrada	Samsung 870 EVO 1TB SATA	11	Carlos	Vega Ramos
12	2026-08-25	Entrada	Samsung 980 500GB NVMe	13	Carlos	Vega Ramos
13	2026-08-25	Entrada	AMD Ryzen 7 5700X	6	Carlos	Vega Ramos
14	2026-08-25	Entrada	Intel Core i5-12400F	6	Carlos	Vega Ramos
15	2026-08-25	Entrada	ASUS Dual GeForce RTX 4060 8GB	5	Carlos	Vega Ramos
16	2026-08-25	Entrada	Corsair CV650 650W 80 Plus Bronze	8	Carlos	Vega Ramos
17	2026-08-25	Entrada	Intel Core i3-12100F	7	Carlos	Vega Ramos

```sql
-- INNER JOIN 8: productos especializados en RAM y sus características.
SELECT p.Nombre_prod, r.Capacidad_gb, r.Velocidad_mhz
FROM dbo.Producto p
INNER JOIN dbo.RAM r ON p.Codbarras_prod = r.Codbarras_prod;
```

	Nombre_prod	Capacidad_gb	Velocidad_mhz
1	Kingston Fury Beast 16GB DDR4 3200	16	3200
2	Kingston Fury Renegade 32GB	32	3600

Diseño fisico

Sentencias DML

FUNCIONES DE COLUMNAS

Estas funciones permiten realizar cálculos sobre los datos almacenados en las columnas, como sumar ventas, contar registros o calcular promedios.

```sql
-- COUNT: cantidad de productos registrados.
SELECT COUNT(*) AS Cantidad_productos
FROM dbo.Producto;

-- SUM: valor del inventario (precio de venta por stock).
SELECT SUM(Precio * Stock) AS Valor_total_inventario
FROM dbo.Producto;

-- AVG, MIN y MAX: resumen de precios.
SELECT AVG(Precio) AS Precio_promedio,
       MIN(Precio) AS Precio_minimo,
       MAX(Precio) AS Precio_maximo
FROM dbo.Producto;
```

	Cantidad_productos
1	10

	Valor_total_inventario
1	29650.00

	Precio_promedio	Precio_minimo	Precio_maximo
1	505.000000	180.00	1450.00

```sql
-- GROUP BY: cantidad de productos y valor del stock por marca.
SELECT m.Nombre_marca,
       COUNT(*) AS Cantidad_productos,
       SUM(p.Precio * p.Stock) AS Valor_stock
FROM dbo.Producto p
INNER JOIN dbo.Marca m ON p.ID_marca = m.ID_marca
GROUP BY m.Nombre_marca
ORDER BY Valor_stock DESC;
```

	Nombre_marca	Cantidad_productos	Valor_stock
1	Kingston	2	7100.00
2	ASUS	1	5800.00
3	Intel	2	5160.00
4	Samsung	3	4880.00
5	AMD	1	4750.00
6	Corsair	1	1960.00

Diseño fisico

Sentencias DML

FUNCIONES DE Fecha

Las funciones de fechas permiten consultar, extraer o calcular información relacionada con fechas registradas en la base de datos.

```sql
-- Fecha y hora actuales.
SELECT GETDATE() AS FechaHoraActual;

-- Extraer año, mes y día de cada venta.
SELECT Numero_comprobante, Fecha_venta,
       YEAR(Fecha_venta)  AS Anio,
       MONTH(Fecha_venta) AS Mes,
       DAY(Fecha_venta)   AS Dia
FROM dbo.Venta;

-- Ventas de septiembre de 2026.
SELECT Numero_comprobante, Fecha_venta, Total_venta
FROM dbo.Venta
WHERE Fecha_venta >= '2026-09-01'
  AND Fecha_venta <  '2026-10-01'
ORDER BY Fecha_venta;
```

	FechaHoraActual
1	2026-10-12 11:27:17.613

	Numero_comprobante	Fecha_venta	Anio	Mes	Dia
1	B001-000001	2026-09-01	2026	9	1
2	F001-000001	2026-09-10	2026	9	10
3	B001-000002	2026-09-22	2026	9	22
4	B001-000003	2026-10-01	2026	10	1
5	B001-000004	2026-10-05	2026	10	5

	Numero_comprobante	Fecha_venta	Total_venta
1	B001-000001	2026-09-01	360.00
2	F001-000001	2026-09-10	1190.00
3	B001-000002	2026-09-22	260.00

```sql
-- DATEDIFF: días transcurridos desde cada venta hasta hoy.
SELECT Numero_comprobante, Fecha_venta,
       DATEDIFF(DAY, Fecha_venta, CAST(GETDATE() AS DATE)) AS DiasDesdeVenta
FROM dbo.Venta;
```

	Numero_comprobante	Fecha_venta	DiasDesdeVenta
1	B001-000001	2026-09-01	41
2	F001-000001	2026-09-10	32
3	B001-000002	2026-09-22	20
4	B001-000003	2026-10-01	11
5	B001-000004	2026-10-05	7

GRACIAS ;)